SAEON Field Data System

Node.js API — Developer Handover

Phase 1: Data Custody

Field	Detail
Project	SAEON Field Data System (saeon-field-data)
Repo	https://github.com/SAEON/saeon-field-data
Prepared by	Samkelo (SAEON)
Date	March 2026
Status	Handover — Phase 1 in progress
Your task	Build the Node.js API inside api/

Important — read before touching any files

Do NOT create new folders. The folder structure for Phase 1 is already defined.

Do NOT add packages beyond what is already installed unless discussed with Samkelo.

The database (fds) is already running locally. All tables are created. Seed data is loaded.

Auth is deliberately deferred. No login system for Phase 1 — build as if all users are trusted.

Your entry point is api/src/index.js — this file does not exist yet. That is where you start.

1. What This System Is

SAEON (South African Environmental Observation Network) operates a network of remote environmental monitoring stations — weather stations, rain gauges, and groundwater sensors. Field technicians physically visit each station, download data from the logger, and optionally record manual readings (e.g. dipper depth for groundwater calibration).

The system being built is a field data collection and custody platform. It replaces a fragmented workflow where data lived on technicians' laptops and was manually uploaded to a shared drive with no record of who uploaded what or when.

The core problem being solved
No central record of what data has been collected or by whom
Manual field readings (dipper depths) recorded in notebooks — never digitised Researchers cannot access data without contacting a technician personally
Duplicate files, no deduplication, no custody trail

Three Data Families

Type	Logger Brand	File Format	Key Detail
Rainfall	HOBO	CSV	Event-based — row per rain tip. Mixed with interference events.
Groundwater	Solonist	XML (.xle)	Fixed interval. Needs manual dipper readings for calibration.
Meteorological	Campbell Scientific	TOA5 .dat	Fixed interval. One station = multiple files (5min, daily, monthly).

2. Repository Structure

Monorepo. One repo, three packages. You work exclusively inside api/.

```
saeon-field-data/
  ├── db/
  │     └── migrations/
  │           ├── 000_create_database.sql
  │           ├── 001_extensions.sql
  │           ├── 002_tables.sql
  │           ├── 003_indices.sql
  │           └── 004_seed.sql
  ├── api/
  │     └── src/
  │           ├── routes/       ← your endpoint handlers go here
  │           ├── parsers/      ← hobo.js, solonist.js, campbell_toa5.js
  │           ├── db/           ← postgres connection + query helpers
  │           ├── middleware/   ← error handling (auth deferred)
  │           └── index.js      ← entry point — START HERE
  ├── pwa/                      ← frontend (separate developer)
  ├── docs/
  ├── .env.example
  └── .gitignore
```

Installed npm packages (api/)
express — web framework pg — PostgreSQL client dotenv — loads .env into process.env
multer — handles file uploads cors — allows PWA to call the API
nodemon — dev only, auto-restarts on file save (devDependency)

3. Database

PostgreSQL. Database name: fds. Already running locally. All tables created, indices applied, seed data loaded. You connect to it — you do not create or modify it.

Tables

Table	What it represents
stations	Physical monitoring locations — Sileza Camp, VASI A1, etc.
station_data_streams	A specific data type from a station — raw_5_min, raw_rainfall, etc.
users	SAEON staff — technicians and managers
field_visits	A technician's visit to a station. THE core entity.
uploaded_files	Raw logger file exactly as uploaded. Never modified.
manual_readings	Field observations — dipper depths, battery readings, site notes
phenomena	Standardised variable definitions — air_temp_avg, rainfall_tot, etc.
raw_measurements	Individual parsed records from uploaded files

Connection (.env)

The .env file lives at the repo root. Copy .env.example to .env. For local dev:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=fds
DB_USER=postgres
DB_PASSWORD=your_local_password
PORT=3000
NODE_ENV=development
FILE_STORAGE_PATH=/var/saeon-data/raw_files
```

DB Connection — api/src/db/pool.js

Create this file first. It is the single pg pool shared across all routes:

```
const { Pool } = require('pg');
require('dotenv').config({ path: require('path').resolve(__dirname, '../../../.env') });

const pool = new Pool({
  host:     process.env.DB_HOST,
  port:     process.env.DB_PORT,
  database: process.env.DB_NAME,
  user:     process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

module.exports = pool;
```

4. What to Build — Phase 1 Endpoints

Eight endpoints. That is all Phase 1 needs. Build them in this order — each one is a foundation for the next.

4.1 Stations

Method	Path	What it does
GET	/api/stations	List all active stations (id, name, display_name, data_family, region)
GET	/api/stations/:id	Single station detail including its data streams

4.2 Field Visits

Method	Path	What it does
POST	/api/visits	Create a new field visit record
GET	/api/visits	List recent visits (optional ?station_id= filter)
GET	/api/visits/:id	Single visit with its files and manual readings

4.3 File Upload

Method	Path	What it does
POST	/api/visits/:id/files	Upload a raw logger file. Store it. Queue parsing.
GET	/api/files/:id/download	Download the original raw file

4.4 Manual Readings

Method	Path	What it does
POST	/api/visits/:id/readings	Add a manual reading to a visit

Upload endpoint — important behaviour

1. Receive file via multipart/form-data (multer handles this)
2. Compute SHA-256 hash of file contents
3. Check uploaded_files table — if hash already exists, return the existing record (no duplicate)
4. Store raw file to FILE_STORAGE_PATH/{year}/{month}/{hash}_{originalname}
5. Insert row into uploaded_files with parse_status = pending
6. Respond to client immediately — do not wait for parsing
7. Parse file in background (worker thread or setImmediate) — update parse_status when done

5. File Parsers

Three parser modules — one per logger format. Each lives in api/src/parsers/. Each receives a file buffer and returns structured data to insert into raw_measurements.

5.1 HOBO CSV — parsers/hobo.js

Event-based rainfall data. Key characteristics:

- Column headers on row 1 — sensor serial number embedded in rainfall column header e.g. "Rainfall (mm) #20102559"
- Each row is either a measurement OR an interference event (Coupler Detached, Host Connected, End Of File) — rarely both
- Date format: MM/DD/YY HH:MM:SS.0
- Rainfall values are cumulative totals — not per-tip increments

Parser output:

```
// parsers/hobo.js — expected return shape
module.exports = async function parseHobo(buffer) {
  return {
    metadata: {
      date_range_start: <Date>,
      date_range_end:   <Date>,
      record_count:     <number>,
    },
    measurements: [
      {
        phenomenon_name:  "rainfall_tot",  // matches phenomena.name
        measured_at:      <Date>,           // UTC
        value_numeric:    0.254,
        value_text:       null,
        is_interference:  false,
      },
      {
        phenomenon_name:  "logger_interference",
        measured_at:      <Date>,
        value_numeric:    null,
        value_text:       "Coupler Detached",
        is_interference:  true,
      }
    ]
  };
};
```

5.2 Solonist XLE — parsers/solonist.js

XML format. Use the xml2js or fast-xml-parser npm package to parse.

- Station name comes from: //Body_xle/Instrument_info/Project_ID and Location fields combined
- Sample rate embedded in: //Body_xle/Instrument_info/Sample_rate
- Phenomena (channels) in: //Body_xle/Ch1_data_header, Ch2_data_header etc.
- Timestamps split across two XML nodes: //Body_xle/Data/Log/Date and /Time
- Typically two phenomena per file: water level (m) and temperature (degC)
- Offset value per channel may need to be added to readings

5.3 Campbell TOA5 — parsers/campbell_toa5.js

CSV with a 4-line ASCII header before data rows:

```
Line 1: "TOA5","StationName","CR1000","12345","OS32","Program","Sig","TableName"
```

```
Line 2: "TIMESTAMP","RECORD","AirTemp_Avg","Rain_Tot","WindSpeed_Avg", ...
Line 3: "TS","RN","degC","mm","m/s", ...
Line 4: "","","Avg","Tot","Avg", ...
Line 5+: actual data rows
```

- Line 1: station name, logger model, serial, OS, program name, table name
- Line 2: column names — map to phenomena.name via phenomena table
- Line 3: units per column
- Line 4: measure type per column (Avg, Tot, Min, Max, Smp)
- TIMESTAMP column is always first — parse as the measured_at value
- One station can produce multiple files with different table names (raw_5_min, raw_daily etc)
- Each table name maps to a station_data_streams row

6. Entry Point — api/src/index.js

This file does not exist yet. It is where you start. Keep it simple:

```
const express = require('express');
const cors    = require('cors');
require('dotenv').config({ path: require('path').resolve(__dirname, '../../.env') });

const app  = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// Routes — wire up as you build each one
app.use('/api/stations', require('./routes/stations'));
app.use('/api/visits',   require('./routes/visits'));
app.use('/api/files',    require('./routes/files'));

// Health check
app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Error handler
app.use(require('./middleware/error'));

app.listen(PORT, () => {
  console.log(`API running on port ${PORT}`);
});
```

Add to package.json scripts

```
"scripts": {
  "start": "node src/index.js",
  "dev":   "nodemon src/index.js"
}
```

Start dev server with: npm run dev

Error middleware — api/src/middleware/error.js

```
module.exports = function errorHandler(err, req, res, next) {
  console.error(err);
  res.status(err.status || 500).json({
    error: err.message || "Internal server error"
  });
};
```

7. Build Order

Build in this sequence. Each step is testable with curl or Postman before moving on.

Step	Task	Test
1	api/src/db/pool.js — DB connection	node -e "require('./src/db/pool').query('SELECT 1').then(r => console.log(r.rows))"
2	api/src/index.js — Express app	curl http://localhost:3000/health
3	api/src/middleware/error.js	Hit a nonexistent route — should get JSON error
4	api/src/routes/stations.js — GET /api/stations	curl http://localhost:3000/api/stations
5	api/src/routes/visits.js — POST + GET /api/visits	POST a visit, GET it back
6	api/src/routes/files.js — POST upload	Upload a real HOBO CSV file
7	api/src/parsers/hobo.js	Parse the uploaded file, check raw_measurements
8	api/src/parsers/solonist.js	Upload a real .xle file
9	api/src/parsers/campbell_toa5.js	Upload a real .dat file
10	GET /api/files/:id/download	Download the original file back
11	POST /api/visits/:id/readings	Add a dipper reading to a visit

8. Key Decisions Already Made

Do not revisit these without talking to Samkelo first.

Decision	What was decided	Why
Database name	fds	Short, meaningful — Field Data System
File storage	Server filesystem (FILE_STORAGE_PATH)	SAEON is publicly funded — no paid cloud storage
Auth	Deferred to Phase 2	Ship Phase 1 without login friction — staff are trusted
GAIA (Nextcloud)	Phase 2 optional mirror via API	Not the primary store — too fragile as a data source
R pipeline (ipayipi)	Not replicated or replaced	Solving custody first — processing is Phase 3
ORM	Raw pg queries	Schema is simple — no Sequelize/Prisma overhead needed
Framework	Express only	No Fastify, no NestJS — keep it readable

What is explicitly out of scope for Phase 1

- Authentication and login
- Data processing (barometric compensation, drift correction, aggregation)
- GAIA / Nextcloud integration
- Public data portal or researcher access
- Automated notifications or alerts
- Any frontend — that is a separate PWA developer

9. Sample Data Files

Ask Samkelo for sample files to test your parsers against. You need at least one real file of each type:

Type	Example filename	Who has it
HOBO CSV	Sileza_Camp_2024_06_12.csv	Sue (technician lead)
Solonist XLE	VASI_A1_20240614.xle	Sue (technician lead)
Campbell TOA5	MCP_VasiScienceCentre_AWS_2024.dat	Samkelo / ipayipi_vignettes repo

The ipayipi_vignettes repo on GitHub (SAEONData/ipayipi_vignettes) has example met station .dat files under met_cs_qa_eg/. These are good for testing the Campbell TOA5 parser.

10. Contact

All questions go to Samkelo. Do not make architectural decisions independently — this system has a specific design rationale.

Person	Role	For questions about
Samkelo	System architect / full-stack lead	Everything — first point of contact
Sue	Technician lead	Field workflows, logger files, station locations
Marc	Data manager	Researcher requirements, data access needs